Volt: Sequential Control of Option Portfolios

research@strategynet.ai

July 2026

Abstract

An entry portfolio fixes the initial option contracts and quantities; management determines the later closes, hedges, rolls, and exercises. Market moves, fills, assignment, expiration, and settlement also change the position and its available actions. Volt formulates management as sequential control over a versioned state/action graph. The compiler resolves semantic actions to listed contracts and supplies a structural legal mask. The runtime applies fills and lifecycle rules, updates cash and inventory, and journals each transition. A policy scores the remaining actions while a declared reward maps P&L, costs, drawdown, capital use, and constraint events to the training objective. This paper defines the control state, transition and reward contracts, the reinforcement-learning loop, chronological evaluation, and fixed-budget comparisons with rule-based management. QUBO enters as a later experiment for allocating rollout or replay batches; contract legality and portfolio accounting remain runtime responsibilities. The current repository supplies the deterministic compiler, graph, scenario, and runtime layers. Environment orchestration, trajectories, rewards, baseline policies, and training form the next implementation stage.

1 Introduction

Portfolio discovery chooses contracts and signed quantities at a decision time. Portfolio control manages the resulting state as option marks, underlying prices, cash, inventory, exercise opportunities, and executable actions change. Holding to expiry, closing at a profit target, rolling a tested short strike, and delta hedging are distinct policies applied to the same initial portfolio.

American options add holder exercise before expiry and short-side assignment events. Physical settlement may replace an option with underlying inventory and a cash posting. A policy therefore carries continuing positions and account consequences from one decision to the next.

The control environment separates financial authority from statistical choice. The compiler and application determine which actions are admissible. The runtime applies contract, execution, exercise, assignment, and settlement rules. The policy assigns scores or probabilities to the admissible actions. The reward definition and path cohort determine the behavior encouraged by training.

The first experiment uses a small action grammar and bounded path set for which policy trees can be evaluated exhaustively. Rule-based management, tree search, and a standard actor–critic algorithm establish the baseline. Rollout allocation and QUBO follow after state transitions, rewards, and accounting agree on those bounded environments.

2 Control state and transition

2.1 State

At decision time t, the policy input is

$$s_t = (G_t, A_t, M_t, p_t).$$

The graph G_t contains the current option positions, underlying and expiry relationships, cash, underlying inventory, market observations, and lifecycle state. The ordered set A_t contains resolved candidate actions. $M_t \in \{0,1\}^{|A_t|}$ is the aligned legal mask. Provenance p_t identifies the compiled program, runtime state, market observations, and assumptions used to construct the input.

The current `StateActionGraphSample` supplies the portfolio graph, runtime cash and inventory context, candidate-action nodes, typed action-to-state edges, rejection reasons, and the compiler mask. The application adds account capital, margin, liquidity, position limits, and broker eligibility before policy normalization.

2.2 Transition

One policy action and one subsequent market observation produce

$$(s_{t+1}, r_t, e_t) = \mathcal{T}_\omega(s_t, a_t, \xi_{t+1}),$$

where ω names the execution, exercise, assignment, settlement, and accounting rules. The observation ξ_{t+1} contains the next market state and external events. The reward record r_t is derived from the transition, and e_t contains immutable evidence.

The implemented runtime applies one action and one market observation to a versioned portfolio state. It validates the action, updates option lifecycle, cash, and underlying units, and returns a journal entry linked to the prior and next state hashes.

A short-put assignment illustrates the required continuity. Assignment can consume the option, debit strike cash, and create long underlying inventory. The next policy decision sees the delivered asset, remaining cash, open options, and a new action set.

3 Actions and legal masks

3.1 Action vocabulary

The implemented candidate grammar contains:

- `AddLeg` for a listed contract selected by explicit or semantic coordinates;
- `RemoveLeg` and `ResizeLeg` for a live position;
- `Wait`; and
- holder-authorized `Exercise`.

Runtime lifecycle actions are `Hold`, `Close`, `Exercise`, `Assign`, and `Expire`. Assignment and expiry enter as external or system events. A roll is represented by a multi-step close and addition plan;

take-profit, stop, hedge, and roll instructions require a planning layer that emits those primitive actions.

Semantic actions resolve at the decision clock. A request for a call in a tenor and delta bucket uses the contract universe and observations visible at that time. The resolved action records the listed contract, quantity, branch identifier, source rule, and artifact hash.

3.2 Mask composition

Let M_t^{compiler} contain contract, portfolio, authority, and lifecycle legality. Let M_t^{account} contain capital, margin, liquidity, and application policy. The policy receives

$$M_t(a) = M_t^{\text{compiler}}(a) M_t^{\text{account}}(a).$$

The composed mask changes after a fill, exercise, assignment, expiration, market halt, or account-state change. Each excluded action retains a reason from the component that rejected it.

The learner observes the mask and scores admissible choices. Deterministic legality remains reproducible when the model or training algorithm changes.

4 Rewards and horizons

4.1 Outcome record

One transition can produce the financial vector

$$r_t = (\Delta\text{equity}_t, \text{cost}_t, \Delta\text{drawdown}_t, \text{capital}_t, \text{turnover}_t, \text{constraint}_t).$$

The current scenario layer supplies fills, costs, journals, marked pathwise P&L, leg attribution, and terminal P&L. Drawdown, capital, turnover, and constraint-event measures require named account and risk components.

A scalar training reward may be

$$R_t = \Delta\text{equity}_t - \lambda_c \text{cost}_t - \lambda_d \Delta\text{drawdown}_t - \lambda_k \text{capital}_t - \lambda_v \text{violation}_t,$$

with coefficients fixed before test evaluation. The vector record remains available for economic reporting even when the learner uses a scalar.

4.2 Episode and learning horizons

An episode can end at portfolio liquidation, the final contract expiry, a risk breach, or the end of the available path. This financial termination rule is separate from the computational return horizon.

For an n-step actor–critic update,

$$\widehat{R}_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j R_{t+j} + \gamma^n \widehat{V}(s_{t+n}).$$

The bootstrap state retains every open option, right, cash balance, and underlying position. Truncating an update therefore changes credit assignment while leaving the economic episode intact.

5 Policy and learning loop

5.1 Policy model

A set or graph encoder maps the current portfolio state to h_t. Each candidate action has embedding $e_t(a)$. A shared policy head computes

$$\ell_\theta(s_t, a) = f_\theta(h_t, e_t(a), h_t \odot e_t(a)) .$$

Masked action probabilities are

$$\pi_\theta(a \mid s_t) = \frac{M_t(a) \exp\{\ell_\theta(s_t, a)\}}{\sum_{a' \in A_t} M_t(a') \exp\{\ell_\theta(s_t, a')\}}.$$

A value head estimates expected discounted return or a return distribution.

The initial neural baseline uses the invariant encoder developed for static portfolio discovery, followed by a parameter-matched graph encoder. Transfer from supervised portfolio ranking is evaluated against training the control encoder from its initialization.

Proximal Policy Optimization with generalized advantage estimation supplies a standard actor–critic baseline [1, 2]. The environment is also tested with tabular or tree-based control on small state spaces, allowing transition and reward errors to be found without a neural policy.

5.2 Training loop

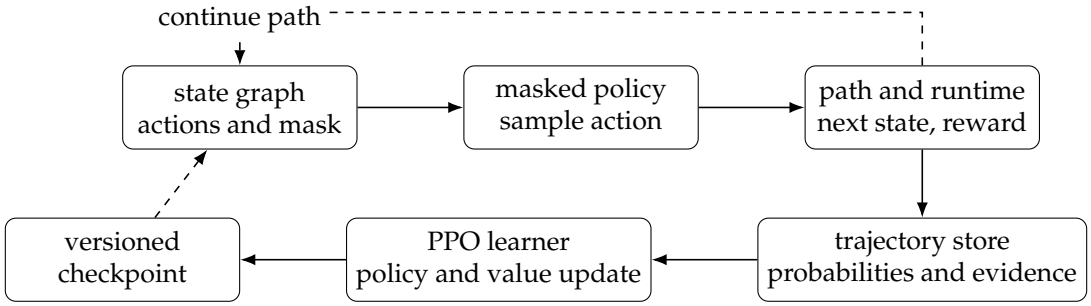


Figure 1: Proposed reinforcement-learning loop. Historical, generated, or forecast-conditioned paths supply market observations; the runtime supplies financial transitions; the learner updates from versioned trajectories.

Figure 1 makes the feedback boundary explicit. Collection workers use one frozen checkpoint for an epoch, run independent path environments, and write immutable trajectory fragments. A deterministic merge orders fragments by stable identifiers before optimization.

Each trajectory step stores state and action identifiers, legal mask, action probability, value estimate, reward components, next-state identifier, termination status, path identifier, runtime journal, and policy version. These fields support replay, off-policy diagnostics, and reproduction of a reported update.

6 Rollout and replay allocation

6.1 Allocation problem

An action tree branches through portfolio decisions, market scenarios, execution outcomes, exercise, and assignment. A fixed simulation allowance may be assigned uniformly, through tree-search rules such as UCT [3], or through an allocator using value uncertainty, tail relevance, novelty, path probability, and estimated simulation cost.

The first comparison uses random allocation, stratified coverage, UCT, greedy utility, and greedy utility with diversity. Exhaustive small trees measure the actions and losses omitted by each allocator.

6.2 QUBO experiment

Let $x_i \in \{0,1\}$ indicate additional computation for frontier item i. For an equal-cost batch of size k,

$$\min_{x\in\{0,1\}^n} -\sum_i u_i x_i + \lambda \sum_{i<j} s_{ij} x_i x_j + \rho \left(\sum_i x_i - k \right)^2 .$$

The acquisition value u_i combines declared signals such as uncertainty, tail relevance, path probability, and estimated improvement. Similarity s_{ij} penalizes a batch concentrated in one region of state or scenario space. Variable simulation costs require a weighted budget formulation.

A recent preprint applies QUBO to Monte Carlo reinforcement-learning episode selection using return and state-coverage terms in a finite GridWorld [5]. Option portfolio control supplies a separate test: American lifecycle events, path-dependent costs, heterogeneous action sets, and variable rollout lengths.

Frontier allocation occurs before outcomes are observed and may reduce simulation cost. Replay allocation occurs after collection and reduces learner cost; prioritized experience replay is its baseline [4]. Selection probability, score components, solver configuration, matrix construction time, solver time, and solution quality are charged to the allocator.

Random or stratified capacity is reserved for exercise, assignment, and tail paths. The quadratic matrix grows as $O(n^2)$, so large frontiers require sparse neighborhoods, clustering, or staged allocation before QUBO construction.

7 Empirical design

7.1 Initial environment

The initial environment contains American-style options on one underlying, a bounded position size, a small action grammar, fixed decision cadence, and declared historical or generated path cohorts. The mark or pricing adapter, fill convention, fees, exercise policy, assignment process, account constraints, reward coefficients, and termination rule are shared by every policy.

Small instances retain exhaustive action trees. Development then increases one dimension at a time: decision horizon, number of contracts, action count, scenario count, execution uncertainty, and path length.

7.2 Management baselines

Economically interpretable policies include hold to expiry, fixed take-profit and stop rules, clock-based hedging, delta-threshold hedging, and a declared roll convention. Learning comparisons include tabular control, tree search, PPO, and the same policy with alternative set or graph encoders.

Measurements include:

- net and gross P&L, drawdown, tail loss, turnover, and capital use;
- exercise, assignment, settlement, and constraint-event coverage;
- stability under alternative costs, spreads, path sources, and assignment rules;
- value calibration and bootstrap error by remaining horizon;
- simulator calls, expanded nodes, environment steps, learner updates, and elapsed time; and
- allocation overhead and selected-state diversity.

Equal simulator calls measure statistical efficiency. Equal elapsed-time budgets include batching, environment, and allocation overhead.

7.3 Time split and policy evidence

Training, validation, and test periods follow the decision clock. Overlapping outcome horizons are assigned to one split or separated by a purge interval. Reward coefficients, stopping rules, exploration share, checkpoint selection, and rollout budget are fixed using training and validation data.

The test report retains gross and net results, path-level outcomes, policy and environment seeds, failures, legal-mask violations, exercise and assignment counts, and comparisons with the rule-based policies. A path cohort gives results under its recorded historical observations or generated model.

8 Current implementation and development order

Volt currently supplies exact option contracts, European/Bermudan/American exercise schedules, portfolio netting, semantic hashes, immutable runtime state, exercise and assignment transitions, cash and underlying emissions, versioned historical or generated market observations, ordered scenario paths, scenario fills and costs, pathwise P&L, compiler action sets, graph samples, and structural legal masks.

A static research driver now supplies generated cohorts, American CRR marks, fixed exercise and assignment decisions, invariant portfolio tensors, and evidence packets. Sequential control additionally requires:

1. an environment adapter that combines those cohort and mark components with compiled actions and the scenario and lifecycle runtimes;

2. named account constraints, hedge actions, reward components, termination rules, and transition evidence;
3. a versioned trajectory store and deterministic replay command;
4. rule-based policies and exhaustive small-tree evaluation;
5. PPO with generalized advantage estimation and the invariant portfolio encoder;
6. the relation-aware graph comparison; and
7. rollout baselines followed by the QUBO allocator.

The planned learner and replay adapter uses Jormungandr's external-actor boundary, split-aware experience records, and replay services. Volt retains the financial transition and legal-action authority; Jormungandr receives versioned observations, masks, actions, rewards, and next states after the environment passes deterministic replay and rule-policy checks.

9 Conclusion

Sequential option management is defined by continuing state, admissible actions, financial transitions, and a declared reward. The Volt compiler and runtime supply contract and lifecycle authority; the policy chooses among the actions that remain; the trajectory connects each choice to its fills, cash, inventory, and outcome.

The first development target is a bounded replayable environment with rule-based and exhaustive controls. PPO and graph encoders follow on the same environment contract. QUBO is reserved for a fixed-budget allocation comparison after conventional rollout and replay methods have established the baseline.

References

[1] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, International Conference on Learning Representations, 2016, https://arxiv.org/abs/1506.02438.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017, https://arxiv.org/abs/1707.06347.

[3] L. Kocsis and C. Szepesvári, *Bandit Based Monte-Carlo Planning*, European Conference on Machine Learning, 2006, https://doi.org/10.1007/11871842_29.

[4] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, *Prioritized Experience Replay*, International Conference on Learning Representations, 2016, https://arxiv.org/abs/1511.05952.

[5] H. Salloum, A. Jnadi, Y. Kholodov, and A. Gasnikov, *Quantum-Inspired Episode Selection for Monte Carlo Reinforcement Learning via QUBO Optimization*, 2026, https://arxiv.org/abs/2601.17570.